

Word specification checklist

Changes so ECO5040W_Business_And_Tech matches the code

ECO5040W Group 3 · 27 September 2026

Each item was checked against the current code. Suggested wording is in italics; paste it in or adapt it. Tick items off as you go.

1 Clean up first

- ☐ Delete “Hi Karabooooo” and the image under the settlement section.
- ☐ Delete comments KIW1 and KIW2.
- ☐ Renumber: after “4. Regulatory Considerations” the headings restart at 2 and 3. Make User journey 5, Functional requirements 6, Fee model 7, Exchange rates 8, Limits 9, Architecture 10, Cash-in 11, Settlement 12, Cash-out 13, Database 14, API 15, Security 16, Regulatory 17, Assumptions 18.
- ☐ Fix cross-references after renumbering: FR-13 “(Section 5)” → Exchange rates; FR-15 “(Section 4)” → Fee model; FR-16 “(Section 6)” → Limits; FR-04 “(Section 13)” → Assumptions and limitations (13 is currently Security).
- ☐ Headings: remove the stray “3” in “3.1 3Registration”; add spaces in “2.3Wallet”, “3.3Beneficiary”, “3.6Queue”, “3.7Recipient”; remove the semicolon in “Regulatory considerations;”.
- ☐ Make “Table 11” match the actual table count.

2 Business sections

- ☐ **Business problem, last paragraph.** Add after it:

In the prototype, RLUSD is stood in for by UCTUSD, a course-issued test token on the XRP Ledger Testnet, treated as worth one US dollar.

- ☐ **3.4 Summary.** “Targets a total cost below that of the digital providers” is false at the default fees (about 6.8% on R1,000, about 5.2% at the R3,000 daily cap). Replace the red sentences with:

The platform competes on this ground. It delivers a complete service from rand to cash, and the recipient can hold the funds as dollar value rather than cash out immediately. At its default fees (R25 plus 1%, a 2% exchange-rate margin and a 1.5% cash-out fee) the total cost is about 5.2% at the R3,000 daily cap and higher on smaller amounts. That is close to WorldRemit but not below it, and not under the 3% target. The fees are configurable, and lowering them is a business decision rather than a technical one.

- ☐ **4.1 KYC and AML.** Replace the red paragraph with:

The prototype collects mock KYC details, blocks sending and cashing out until an administrator approves the application, and enforces daily and monthly rand caps. These are not a transaction-monitoring or reporting engine: nothing screens sanctions lists or files reports with the Financial Intelligence Centre.

- ☐ **4.2 Licensing, last sentence.** Replace “The platform therefore treats RLUSD transfers like any other transfer and applies the same limits” with:

A licensed dealer would have to apply the annual allowance to these transfers. This prototype does not enforce it; it enforces only its own configurable daily and monthly caps (R3,000 and R25,000 for a verified sender).

- ☐ **4.3 Consumer protection.** Replace “the exact amount their recipient will receive” with:

...the exchange rate, and an estimate of what their recipient will receive. The cash-out fee and payout are estimates, recalculated when the recipient actually cashes out.

□ **4.4 Custody.** Replace the red paragraph with:

Because the platform holds the private keys to each recipient's custodial account, it carries responsibility for keeping those assets safe. In the prototype, one platform treasury pays each recipient's custodial Testnet account, and each user's spendable balance is an internal ledger. There is no separate bank account for client money, since cash-in and fiat payout are simulated. Private keys are stored encrypted, the decryption key is kept in the environment rather than the database, and a key is decrypted only immediately before settlement, cash-out completion or a TrustSet signs.

3 Functional requirements: requirement text

□ **FR-12b and FR-12c are swapped** relative to the brief. Swap both columns:

- FR-12b: *Provision a custodial wallet only once the beneficiary is linked to a registered recipient account.* Implemented in: *A wallet row is created when the beneficiary is linked; the Testnet account itself is created at the first settlement, and only for that linked user.*
- FR-12c: *If no account matches, leave the beneficiary unlinked; no wallet and no settlement until a matching account registers.* Implemented in: keep the current “The record stays unlinked...” text.

□ **FR-19.** Change the requirement to *Administrator confirms the payment.* Add to Implemented in: *There is no mock payment-provider route; the brief's “or” is satisfied by the administrator.*

4 Functional requirements: “Implemented in” cells

FR	Replace or append
□ FR-06	<i>POST /kyc overwrites a pending or rejected application and returns 409 once it is approved. The screen locks the form while it is pending.</i>
□ FR-08	<i>POST /kyc/{id}/approve or /reject. A missing or blank rejection reason returns 422; the UI asks for one and confirms the decision in a dialog.</i>
□ FR-09	<i>POST /remittances returns 403 without approved KYC; the send screen shows “KYC not approved” with a line for pending, rejected or not submitted.</i>
□ FR-10	Append: <i>payout_currency</i> is stored and shown, but quotes do not use it.
□ FR-12a	Append: <i>Mobile or email is enough; the first matching account wins. Editing a profile does not relink. Email matching is case-insensitive.</i>
□ FR-14	Append: <i>estimated_recipient_payout</i> is in UCTUSD, not rand, and is an estimate.
□ FR-16	<i>Checked on the gross rand amount before the quote is inserted; refused only when usage plus the amount is strictly over the cap. Cancelled and expired quotes do not count. Two simultaneous quotes are not locked, so both can pass.</i>
□ FR-22	Append: <i>Each call drains the stream once; the worker does not stay running between calls.</i>
□ FR-25	Append: <i>Before submitting, the worker looks up a validated Payment whose memo is this remittance id and adopts that hash instead of paying again. Two overlapping worker runs can still both miss the lookup and pay twice.</i>
□ FR-26	Append: <i>If the process dies before that commit, the next run adopts the payment by its memo and then credits the wallet.</i>

	FR	Replace or append
☐	FR-28	Change the on-chain part to: <i>The on-chain figure is spendable plus requested or approved cash-out amounts; it is not a live read from the ledger.</i>
☐	FR-28a	<i>GET /remittances/me lists amount, status and tracking reference on /app/history. The quote breakdown, status timeline, hash and QR code are on /app/track/{ref} and the quote page.</i>
☐	FR-32	<i>Complete burns the gross reserved amount by paying the issuer; the fiat payout is the net after the fee and no bank payment is made. Fail refunds the gross amount.</i>
☐	FR-35	Append: <i>Fail refunds the cash-out user (the recipient), not the remittance sender. Each action is confirmed in a dialog.</i>

5 Fee model

- ☐ In “the sender receives fewer dollars”, change **sender** to **recipient**.
- ☐ Replace the rounding sentence with:

The rand transaction fee and the fiat payout are rounded to two decimal places; token amounts and the cash-out fee to six. Halves round up.

- ☐ Add a worked example after the formulas:

R1,000 at the 18.50 fallback rate: fee R35.00, effective rate 18.87, net R965.00, 51.139375 UCTUSD, estimated cash-out fee 0.767091, estimated payout 50.372284 UCTUSD.

6 Exchange rates

Correct against the code, including the five-minute fallback retry. No change needed.

7 Limits

- ☐ “depend on whether is verified” → *depend on whether the sender’s KYC is verified.*
- ☐ Table header: **Tier | Daily limit | Monthly limit** (“Montly” is misspelled).
- ☐ Replace “the zero limits exist such that unverified users cannot send” with:

Unverified senders are refused earlier, with a 403 for missing KYC, so they never reach the R0 check. R0 is the configured tier shown on GET /limits/me.

- ☐ In the “Everything else counts” paragraph, add:

Usage sums the gross zar_amount. A quote is refused only when usage plus the new amount is strictly over the cap, so a send that lands exactly on the cap is allowed.

- ☐ Status table: change the row “quoted (past TTL)” to *quoted, after expiry*, with *cash-in returns 409 once expires_at has passed* in the next column.

8 Architecture diagram

- ☐ Add prose under the diagram:

Users reach one React web app, which calls the FastAPI platform API with a bearer token. The API reads the live USD/ZAR rate, stores all state in SQLite, and on a confirmed cash-in adds a message to a Redis stream. The settlement worker reads that stream and pays UCTUSD from the platform treasury to the recipient’s custodial Testnet account, then records the hash

and credits the balance in one commit. The prototype uses both wallet options in the brief: a real Testnet account per linked recipient, and an internal spendable balance. The on-chain figure shown in the wallet is not a live ledger read.

9 Database design

- ☐ Check the KYC status list in the entity diagram matches the code: `not_submitted | pending | approved | rejected`.
- ☐ Add:

The platform wallet, fee settings and limit tiers stand alone, as shown in the second diagram. The schema is built by three Alembic migrations (0001–0003).

10 Already aligned

Cash-in, settlement, cash-out, API, security, regulatory (prototype) and assumptions were written by Karabo against the code. They only need renumbering.